

Rapport de projet

ESA AI Embassador

Quentin Ruel et Olivier Baudouin – M1 ESA 25/26

Introduction :

Ce rapport documente la conception, le développement et l'optimisation d'un assistant conversationnel intelligent destiné à répondre aux questions concernant la formation des étudiants en recherche de master et ceux actuels du Master ESA. Le système, baptisé ESA IA, repose sur une architecture de type Retrieval-Augmented Generation (RAG) combinant une base de connaissance structurée en Markdown, une base vectorielle ChromaDB pour la recherche sémantique, le modèle d'embedding multilingue `infloat/multilingual-e5-large`, et le modèle de langage Llama-4-Scout accédé via l'API Groq.

Le développement a suivi une démarche itérative et empirique : à partir d'une première version fonctionnelle, nous avons réalisé plus de 60 tests systématiques qui ont permis d'identifier et de corriger les défauts de récupération, de génération, de présentation et d'expérience utilisateur. L'aboutissement est un chatbot fonctionnel accessible via une interface Chainlit personnalisée aux couleurs du master, intégrant un système de feedback utilisateur connecté à Google Sheets pour permettre une amélioration continue.

Dans ce rapport de projet, nous allons détailler la démarche que nous avons suivi pour la mise en place de ESA IA étape par étape, dans l'ordre chronologique de nos travaux.

1. Objectifs du chatbot :

L'objectif principal du projet est de concevoir un assistant conversationnel intelligent capable de répondre de manière naturelle et fiable aux questions formulées par les visiteurs du site et qui serait, à terme, déployable sur le site du master. Nous voulions qu'il soit capable de répondre à l'ensemble des informations relatives au master : présentation générale, programme du M1 et du M2, débouchés et insertions, candidatures, etc.

Afin d'éviter au maximum les hallucinations, nous avons utilisé l'approche du RAG qui permet, à partir de la question de l'utilisateur, d'identifier les passages les plus pertinents dans la base de connaissances vectorielle (retrieval) et de transmettre ces passages récupérés à un LLM accompagné d'instructions précises. Ce LLM rédige ensuite une réponse en s'appuyant EXCLUSIVEMENT sur ces extraits.

Cette architecture présente 3 avantages majeurs pour notre cas d'usage :

- La base de connaissances peut être mise à jour sans réentraîner le modèle
- Les réponses sont traçables (on sait quels passages ont été utilisés)
- Le risque d'hallucination est très fortement réduit

2. Construction de la base de connaissances :

La base de connaissance est constituée de fichiers Markdown organisés thématiquement. Ce format a été retenu pour plusieurs raisons : il est lisible humainement, facilement éditable, structurable hiérarchiquement et bien supporté par les bibliothèques de traitement de texte. Surtout, il facilite la phase de chunking en exposant explicitement la structure du document via ses titres.

La base comprend 33 fichiers Markdown qu'on peut répartir en 6 catégories :

- **Présentation générale** : un seul fichier .md
- **Programme détaillé** : un fichier .md par semestre, contenant pour chaque matière (si disponible sur le site du master au moment du scraping) l'enseignant, les ECTS, le volume horaire, les prérequis, le plan du cours et la bibliographie
- **Candidature et admission** : un fichier .md décrivant la procédure, le calendrier et la capacité d'accueil
- **Insertion professionnelle** : un fichier .md comprenant les statistiques d'insertion par promotion
- **Voie recherche** : un fichier .md détaillant le cursus recherche et la liste des personnes l'ayant suivie
- **Annuaire des alumni** : 22 fichiers .md (un par promotion de 2005 à 2026), contenant les noms, poste, employeur et LinkedIn d'un total de 642 diplômés référencés.

Pour les Markdown des 5 premières catégories, nous avons simplement scrapé les données disponibles sur le site du Master ESA. En ce qui concerne l'annuaire des alumni nous avons, à partir d'une liste de tous les diplômés, récupéré à la main les informations en cherchant individuellement tous les profils sur LinkedIn que nous avons rapporté dans un fichier Excel. Nous avons fait du mieux que l'on pouvait, mais les diplômés n'avaient parfois pas de compte LinkedIn (ou nous ne l'avons pas trouvé), et pour d'autres il est possible qu'il ne soit plus actif et donc les informations récoltées ne sont pas à l'ordre du jour.

Un script Python a été développé pour transformer le fichier Excel fini en 22 fichiers .md standardisés, un par promotion. Un cas particulier mérite d'être souligné : la promotion 2026 est constituée d'étudiants encore en cours de scolarité (en stage actuellement). Une correction manuelle a été apportée à leur fiche pour préciser ce statut, afin d'éviter que le chatbot ne les présente à tort comme des diplômés.

Une règle d'or s'est dégagée au fil de la construction de la base : chaque fichier Markdown doit avoir un périmètre thématique unique et non-recouvrant. Initialement, le fichier `master_esa_insertion.md` contenait à la fois des statistiques d'insertion et des informations sur la voie recherche, qui étaient également présentes dans `master_esa_recherche.md`. Cette duplication entraînait deux problèmes :

- Au moment du chunking, deux extraits sémantiquement quasi-identiques étaient créés, ce qui dégradait la qualité de la récupération vectorielle (les deux pouvaient être retournés en doublon ou se concurrencer)
- En cas de mise à jour d'une information, il fallait penser à le mettre à jour dans les deux fichiers

Une phase de nettoyage a donc été conduite pour rétablir des frontières claires entre fichiers. Cette règle s'est par la suite avérée fondamentale pour la qualité globale du système.

3. Chunking :

Pour ce projet, le script `chunk_all.py` a été développé pour implémenter une stratégie de découpage adapté au Markdown. Plutôt qu'un découpage aveugle par nombre de caractères, le script exploite la structure hiérarchique des titres : chaque section (délimitée par `##`) devient un chunk autonome, avec ses sous-sections (`###`) incluses. Si la section contenait trop de caractères, le découpage se faisait au niveau des sous-sections.

Pour améliorer la qualité du retrieval, chaque chunk est préfixé d'un en-tête contextuel indiquant son origine (nom du fichier source, titre de la section). Cet enrichissement améliore significativement la recherche sémantique.

En ce qui concerne les fichiers d'albumis, une stratégie spécifique a été mise en place : chaque diplômé fait l'objet d'un chunk individuel, et un chunk par promotion (contenant la liste complète de la promo). Ce double niveau permet à la fois les recherches ciblées (« qui travaille chez Deloitte ? ») et les recherches globales (« combien d'albumis dans la promotion 2020 ? »).

Le choix du modèle d'embedding (qui transforme chaque chunk en vecteur numérique) est crucial pour la qualité du retrieval. Le modèle retenu est `infloat/multilingual-e5-large` pour les raisons suivantes :

- Taille de contexte généreuse (512 tokens), suffisante pour la majorité des chunks après préfixage contextuel
- Performance de pointe sur les benchmarks MTEB
- Convention de préfixage
- Open-source et téléchargeable localement, pas de dépendance à une API payante

Les vecteurs générés par le modèle d'embedding sont stockés dans une base de données vectorielle locale : ChromaDB. Sa simplicité d'utilisation, son fonctionnement embarqué et ses performances adaptées pour la volumétrie du projet (889 chunks au total) en font une très bonne option.

Au final, le pipeline complet d'indexation, exécuté à chaque mise à jour de la base de connaissances, suit 5 étapes :

1. Lecture des fichiers .md du dossier `markdown_files/`
2. Découpage en chunks selon la structure hiérarchique des titres
3. Préfixage contextuel et génération des chunks de synthèses pour les albumis
4. Embedding
5. Indexation dans ChromaDB avec stockage des métadonnées

4. Intégration du LLM et premier prototype :

Avec la base de connaissance vectorielle en place, l'étape suivante était de choisir un LLM adapté pour la phase de génération. Nous avons opté pour Llama-4-Scout accédé via l'API Groq comme recommandé car elle propose une infrastructure d'inférence très rapide et un tier gratuit suffisant pour le développement et les premiers tests. Le modèle offre en plus une fenêtre de contexte généreuse (128k tokens) et une bonne capacité à suivre des instructions structurées.

Pour l'interface utilisateur, nous avons choisi Chainlit, framework de Python conçu spécifiquement pour les applications conversationnelles à base de LLM. Il offre une interface de chat moderne et responsive, le texte des réponses apparaît progressivement, un système d'actions intégré, une gestion de session utilisateur et surtout un support de personnalisation visuelle via CSS, indispensable pour adapter l'interface aux couleurs du Master ESA.

Nous ensuite créé le premier prototype du chatbot, qui suivait l'architecture classique d'un RAG conversationnel et dont l'interface Chainlit était celle par défaut. À partir de celle-ci, nous avons testé quelques questions et nous avons vite décelé des problèmes de retrieval et de contexte dans les réponses du chatbot. Ainsi, nous avons paramétré petit à petit notre modèle en augmentant le nombre de chunks récupérés, en paramétrant le `max_distance`, en configurant une mémoire pour les 5 derniers échanges (et donné plus de poids au dernier message suite à quelques problèmes de logique), en établissant un prompt système, etc.

Très rapidement nous avons mis en place une distinction entre deux profils d'utilisateurs avec des droits d'accès différents :

- Mode visiteur : accès aux informations publiques uniquement. Pas d'accès à l'annuaire des alumni.
- Mode connecté : accès complet, incluant l'annuaire des alumni et la recherche d'anciens étudiants.

Cette distinction est implémentée via deux mécanismes complémentaires : au moment du chunking, chaque chunk contenant un drapeau «  Informations réservées aux étudiants » se voit attribuer une métadonnée d'accès restreint, et au moment du retrieval si l'utilisateur est en mode visiteur, ces chunks sont filtrés en amont avant même d'être transmis au LLM. Cette approche garantit que les données sensibles ne peuvent jamais être révélées en mode visiteur, même par contournement du prompt : le LLM ne les voit tout simplement pas.

Pour aligner l'identité visuelle du chatbot sur celle du Master ESA, un travail conséquent de personnalisation CSS a été mené. Les éléments suivants ont été adaptés :

- Bandeau supérieur avec le logo du master (versions claire et sombre)
- Titre « ESA IA » centré dans le bandeau
- Avatar du bot remplacé par le bouclier ESA (versions claire et sombre), affiché à gauche de chaque réponse (et les réponses uniquement, sinon ça faisait trop chargé)
- Bulles de discussions et zone de saisie personnalisées en fonction du thème
- Modes clair et sombre entièrement supportés avec des palettes cohérentes
- Favicon (icône sur l'onglet navigateur) personnalisé en bouclier ESA

C'était un long travail itératif : en inspectant le DOM via les outils développeur du navigateur (F12), il a fallu identifier les sélecteurs CSS adéquats pour chaque élément. Nous avons ensuite ajouté les personnalisations petit à petit, regardé le résultat, en avons fait d'autres, sommes revenus en arrière... Mais nous sommes pleinement satisfaits du résultat final qui correspond exactement ce que nous imaginions.

À l'issue de cette phase, le chatbot était fonctionnel : il répondait aux questions sur le master, distinguait les deux modes d'accès, citait ses sources et présentait une interface visuelle aboutie. La question naturelle suivante était : à quel point répond-il bien ? C'est pour répondre à cette dernière qu'une démarche d'évaluation systématique a été mise en place.

5. Démarche d'évaluation et identification des défauts :

Nous avons conçu une batterie de tests visant à couvrir tous les types de requêtes susceptibles d'être posées (en pratique) par un utilisateur réel. Au total, plus de 60 questions ont été élaborées, classées en grandes catégories :

1. Présentation / structure du master
2. Candidature et prérequis
3. Programmes / matières
4. Stages
5. Insertion / débouchés
6. Promotions
7. Alumni
8. Contact
9. Recherche / poursuite d'études
10. Hors-scope

À partir de tous ces tests effectués, nous avons pu cibler certains problèmes que rencontrait le chatbot. Nous allons en détailler quelques-uns et les solutions adoptées ci-après.

Le mode visiteur trop restrictif : il y avait un filtrage trop agressif en mode visiteur. Initialement, dès qu'une question contenait le mot « étudiants » ou « promo », le système considérait qu'il s'agissait d'une demande sur les alumnis et bloquait l'accès. Or ces termes apparaissent légitimement dans des questions publiques comme « combien d'étudiants par promotion ? » par exemple. La détection a donc été affinée.

Un deuxième défaut concernait l'ordre des éléments dans les listes. Pour les questions sur les alumnis (« qui travaille chez Deloitte ? »), il est attendu que les diplômés soient présentés du plus récent au plus ancien. Pour amener le LLM à respecter cet ordre, une règle explicite a d'abord été ajoutée au prompt système : « Quand tu présentes des informations qui concernent plusieurs promotions ou plusieurs années, tu DOIS classer du plus récent au plus ancien ». Cette règle a eu un effet secondaire imprévu : elle s'appliquait également aux listes de matières d'un semestre, ce qui n'a aucun sens. Une seconde itération a précisé la règle pour qu'elle ne s'applique que pour les données indexées par année ou par promotion avec des exemples explicites.

Un autre défaut récurrent concernait les questions du type « qui travaille chez X ? ». Le retrieval renvoyait bien quelques chunks pertinents, mais le LLM produisait souvent des réponses incomplètes. Plusieurs hypothèses ont été investiguées :

- TOP_K trop faible
- Distance mal calibrée
- Le LLM lui-même qui « résume » au lieu de tout lister

L'analyse a montré que les 3 hypothèses étaient partiellement vraies. Pour palier ce problème, nous avons mis en place une détection automatique du nom d'entreprise combinée à une recherche exacte par métadonnées sur les chunks alumnis pour récupérer tous les diplômés concernés. De plus, nous avons injecté dans le prompt une liste pré-triée par promotion descendante avec une instruction explicite : « Cette liste contient EXACTEMENT N alumnis, ne pas en omettre, vérifier le compte avant de répondre ».

Cette approche, qualifiée de « boost entreprise », garantit une réponse exhaustive sur les entreprises identifiées. Elle a ensuite servi de modèle pour traiter d'autres cas similaires.

Un autre test a révélé que le chatbot interrogé avec « peux-tu m'aider en code ? » répondait par l'affirmatif et proposait des exemples de script en R et en Python. Ce comportement est inapproprié pour l'usage voulu de ce chatbot. Nous avons donc émis une règle de cadrage strict dans le prompt système, listant explicitement les types de demandes à refuser et fournissant un format de refus poli et redirecteur.

Une fois satisfaits de notre chatbot sur la grande majorité des questions testées, nous nous sommes occupés de détails techniques comme enlever la liste des sources consultées (qui n'ont pas forcément servies à rédiger la réponse) pour ne garder que celles utilisées, ou le lissage des sources pour qu'elles soient plus parlantes qu'un nom de fichier .md. De plus, nous les avons regroupées dans un menu déroulant cliquable pour améliorer l'expérience utilisateur et les avons associées à des liens cliquables vers le site du Master ESA, qui mènent aux sections correspondantes.

6. Système de feedback :

Même après une phase de tests approfondie, un chatbot reste un système probabiliste qui peut produire des réponses non-pertinentes ou partielles sur certaines requêtes. La mise en production nécessite donc un mécanisme permettant de collecter les retours utilisateurs et d'identifier les éventuels défauts résiduels une fois le système exposé à un public réel.

Nous avons donc mis en place la fonctionnalité d'évaluation de la réponse du chatbot dans le système, qui donne la possibilité à l'utilisateur de donner un pouce vers le haut ou vers le bas en fonction de sa satisfaction de la réponse qui lui a été donnée. Il nous fallait désormais un moyen de stocker tout cela.

Notre choix s'est porté vers Google Sheets pour stocker les feedbacks pour plusieurs raisons :

- Accessibilité immédiate
- Analyse facilitée
- Partage simplifié
- Coût nul
- Compatibilité

L'écriture dans Google Sheets passe par l'API officielle, via la bibliothèque Python gspread combinée à l'authentification OAuth 2.0 via google-auth. Un compte de service Google Cloud a été créé pour permettre au chatbot d'écrire dans la feuille sans interaction utilisateur. La clé JSON du compte de service a été stockée localement et la feuille est partagée en mode Editeur avec l'adresse email du compte de service.

Pour optimiser les performances, la connexion à la feuille est mise en cache globalement : la première écriture initie la connexion, les suivantes la réutilisent.

La Google Sheet contient 5 colonnes :

1. **Timestamp** (date et heure du vote au format ISO)
2. **Mode** (visitor ou logged_in)
3. **Question** (telle que saisie par l'utilisateur)
4. **Réponse** (le contenu généré par le LLM)
5. **Vote** (👍 ou 👎)

Ces données permettront, une fois suffisamment de volume collecté, d'analyser le taux de satisfaction global, une analyse par mode d'accès, d'identifier les questions problématiques et bien d'autres choses.

7. Déploiement et limites :

Le chatbot a été conçu pour être déployable sur n'importe quelle machine disposant de Python 3.12. La procédure de déploiement se déroule en six étapes :

1. Installation de Python 3.12 (les versions 3.13+ ne sont pas compatibles avec Chainlit au moment de la rédaction)
2. Copie du projet et création d'un environnement virtuel dédié
3. Installation des dépendances
4. Régénération de la base ChromaDB (en théorie pas nécessaire, tout est déjà fait dans le dossier zippé)
5. Lancement de l'application

Les détails de la procédure sont rédigés dans le README.txt à lire impérativement (et dans le word protocole_a_suivre.docx de manière plus humaine, peut-être à préférer pour s'assurer de suivre le bon procédé).

Malgré son bon fonctionnement, le système actuel présente plusieurs limites :

- **Dépendance à une API externe.** En cas de panne ou de modification de l'offre commerciale, le service est interrompu. Une bascule vers une API alternative ou vers un modèle local serait possible mais demanderait une grosse adaptation.
- **Mode connecté simulé :** actuellement, l'utilisateur choisit lui-même son mode (visiteur ou connecté) via un bouton, sans authentification réelle. Cela suffit pour la phase de test et pour le cadre du projet mais devra être remplacé par une intégration avec le système d'authentification du site du Master ESA avant une mise en production publique.
- **Base de connaissance statique :** toute modification (nouvelle promotion d'album, changement de programme, mise à jour des chiffres d'insertion) nécessite une régénération manuelle de la base via les scripts de chunking. Un système automatisé (synchronisation depuis le site officiel, watch des fichiers source) serait souhaitable à terme.
- **Limites de l'API Groq :** le tier gratuit impose une limite de tokens qui bloquent le chatbot lorsqu'elle est atteinte, et qui est très lente à se recharger (ce qui s'avère être très frustrant quand on développe le chatbot). Une montée en charge significative nécessiterait de souscrire à un plan payant ou de basculer vers une autre solution d'hébergement.

Conclusion :

Ce projet aura permis de concevoir, développer et raffiner un assistant conversationnel intelligent au service du Master ESA. Le résultat final est un chatbot accessible, fiable et bien intégré à l'identité visuelle du master.

L'approche itérative et empirique adoptée tout au long du projet s'est avérée fondamentale. Plutôt que de viser une architecture parfaite dès le départ, un premier prototype fonctionnel a été mis en place rapidement, puis confronté à une batterie de tests systématiques. Chaque défaut identifié a donné lieu à une analyse de cause, à une solution ciblée, et à une vérification que la correction n'introduisait pas de régression. Cette méthode, courante dans le développement logiciel mais apparemment moins systématique dans les projets IA a permis de transformer un prototype prometteur mais fragile en un système robuste et maîtrisé.

Le chatbot ESA IA constitue à la fois un livrable utilisable et un terrain d'apprentissage. Il pourra, nous pensons, servir de base pour les futures personnes amenées à travailler sur l'intégration d'un chatbot sur le site du master.

À titre plus personnel, ce projet aura été l'occasion pour nous de découvrir en profondeur l'écosystème des systèmes RAG modernes, et de constater à quel point le passage d'une démonstration impressionnante à un système réellement fiable nécessite une rigueur méthodologique et une attention aux détails qui constituent le vrai cœur du métier d'ingénieur en IA appliquée.